

Lab 01: Introduction to Web Engineering

Objectives:

To understand:

- How does web works
- The basics of web
- How a webpage is rendered
- Basics of html and how to write a basic html page

How Does Web Works?

Introduction

For many of us, the Internet and Web Browsing has become a daily activity. Whether it is for checking stock prices, buying food, doing work, ordering books and music, or just to browse a favorite site, web browsing has become an institution in our lives much the way television is. Have you ever wondered how this whole web thing works, though? This lab is designed to explain the history and concepts of the Web and how it works technically. After you browse to a site, you will understand actually how it is done and how your computer retrieves this information.

The Technology behind the Web

The web works on three standards. These standards are generally adhered to by all companies that make products that work with the World Wide Web.

These standards are:

URL (Uniform Resource Locator): These are the addresses that you enter into your web browser to connect to a web site. The URL is broken up into 4 parts which are the protocol, the hostname, the port number, and the path that you are requesting.

Protocol:

The protocol part of an URL is the funny string of characters that you see before the hostname. Examples are http, ftp, telnet:, etc. They are separated from the hostname with a colon and two forward slashes (://). These protocols tell your browser what type of service to use when you connect with the web browser to the hostname.

If you leave the protocol off your address, by default the Web Browser will assume you are using the HTTP protocol, which is for connecting to web sites, so there is no need to type in the http:// every time you go to a web site. If you specify another protocol like ftp, then the browser will act as an ftp client that will enable you to connect to a ftp server to download files.

Hostname:

The hostname is the address you are going to. For example, if you are going to the address <https://www.bleepingcomputer.com>, then www.bleepingcomputer.com is the hostname.

Port Number:

The port number is a number that you can append to the hostname with a colon (:) between them. For example <https://www.bleepingcomputer.com:80>. If you leave the port number off, which almost everyone does, then the browser will automatically use port 80 as that is the default port for the http protocol.

Path:

This is the path on the server, culminating with the filename you are trying to reach. For example, the URL <https://www.bleepingcomputer.com/examples/example1.html>. The path in this case is `/examples/example1.html`. This path corresponds to an actual directory structure on the web server. So on the web server there is a root directory, an examples directory underneath that root directory, and a file called `example1.html` underneath that.

HTTP (Hyper Text Transfer Protocol):

This is a defined process of how to transfer information between a web browser and a web server. All web browsers and web servers follow this process.

HTML (Hyper Text Markup Language):

This is the language used in web pages to format text, images, and page layout. This language is in pure text and is entered into a file that has an ending of `.html`. It is possible to put HTML in documents that do not end in `.html`, but for the purpose of this tutorial, we are only focusing on pure HTML documents. The text in these documents contain special codes, called tags, that tell the web browser when it reads the file how to format the text. Lets try an example below.

If you were to create a file called `helloworld.html` and save it on your hard drive, you could then open this file with your browser and have it displayed. The contents of this file will have the following text:

Hello World!!!!

If you were to open up this document in your browser you would see the following:

Hello World!!!!

As you can see the text, Hello World, has been shown to you in bold print. This was because we enclosed the words in the tags , **which means any text after it will be bold, and then the ending** means this is the end of the bold formatting. All tags in HTML have a beginning tag, that starts the formatting, and an ending tag, that stops the formatting. There are many many more tags available to use in HTML, the bold (`<b>`) tag being just one of them.

Web Browser and Web Servers

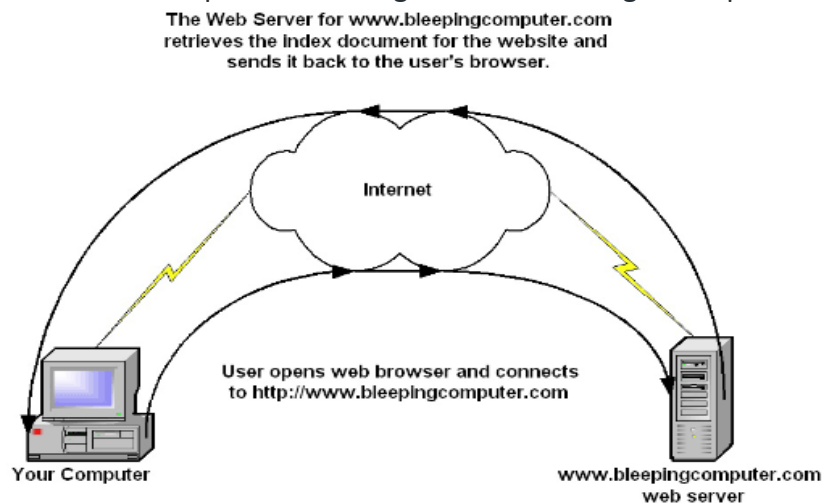
In order for the Web to work you need web browsers and web servers which work hand in hand. The **web browser** is a piece of software that is used to interpret the information found in an HTML document and display the content of that document based upon the HTML tags found within it. A **web server** is a computer that stores HTML documents, otherwise known as web pages, and waits for connections from web browsers. When a web browser connects to a web server, the web server sends the requested document, if it exists, back to the web browser for display.

Actually Browsing a Web Site

Now that you understand the basics behind how the Web works, let's walk you through the actual process of how your computer goes to a web site and displays it in your browser.

The first step of course is to open your web browser, whether that be Chrome, Internet Explorer, or Mozilla. When your browser opens, you have the option of connecting to another web site. In the address field, type the location of where you would like to go. For this example, let's go to www.bleepingcomputer.com.

You type <https://www.bleepingcomputer.com>, or www.bleepingcomputer.com as the `http://` is optional, in the address field and press enter or go. The below diagram explains what happens:



As you can see, when you try to connect to a site, your web browser opens an Internet connection and tries to connect to the web server specified in the host portion of the URL. If it connects, the web browser sends the web server the path portion of the URL. If that path exists on the web server, the web server sends the content of the HTML file back to your browser. Your browser reads through the HTML of the document, following the instructions found there as it displays the information on your screen.

That is all there is to it to retrieving a web page from a remote computer.

What is a Web Page?

A web page or webpage is a document commonly written in HyperText Markup Language (HTML) that is accessible through the Internet or other network using an Internet browser. A web page

is accessed by entering a URL address and may contain text, graphics, and hyperlinks to other web pages and files.

Rendering

How a webpage is loaded and displayed



What are the steps of a webpage being displayed?

1. A request is made when a link is clicked.
2. The page and its resources (files) are downloaded.
3. The web browser uses the page resources to build the page.
4. The page then is rendered (displayed) to the user.

Each of the above steps have many components and details, but those four steps are the main things that happen to display a webpage to a user.

The four steps listed above are often referred to more technically as...

1. Request
2. Response
3. Build
4. Render

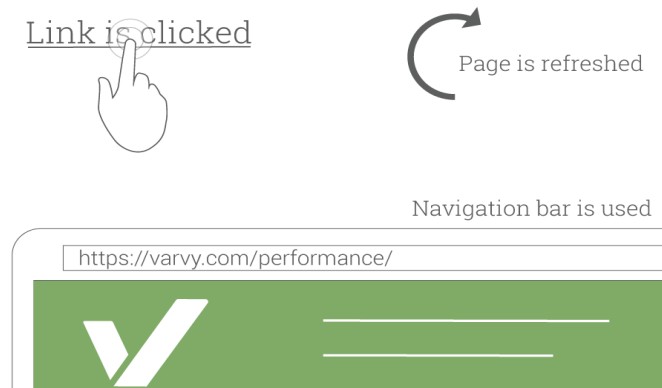
Request	Response
Build	Render

Each of these steps are often performed several times during a page load.

The request

Before we go into what happens when a webpage is requested, let's look at what makes a webpage get requested in the first place.

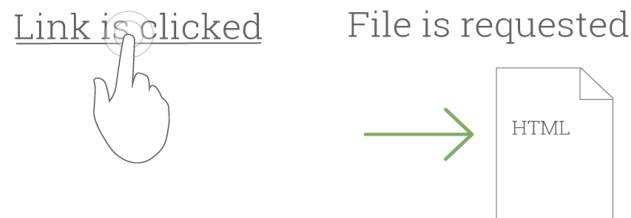
The most common way a webpage is requested is when a link is clicked, but requests also occur when a page is refreshed, or when a url is typed into a browser.



The moment when a webpage is requested is also called "navigation start". It is basically the moment where the whole process of displaying a page begins.

The document

When a linked is clicked a request is made for a document.



The document is the webpage file, which is just a text file. It is located on a web server (a webhost).

This file is often of the type ".html", but the type of file it is doesn't really matter. The important thing is that a text file is requested by the web browser.

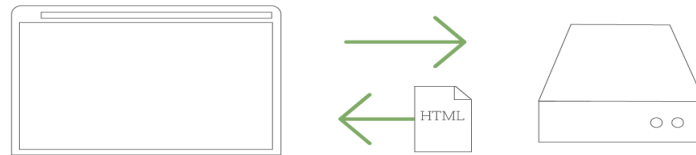
The file gets requested using a system called HTTP. At this point we do not need to understand HTTP, we just need to understand that a web browser asks for the file from a web server.

The response

The web server then provides the file to the web browser. The response is simply the browser receiving the thing it had requested.

1. web browser requests document (html file)

2. web server provides document (html file)



If this were a simple webpage (only HTML) the request and response stage of the page load would be complete. However, most webpages have things like images, css, and javascript. These are called resources and in order to display the webpage the web browser must get those page resources as well.

Parsing

So how does a web browser know if a page needs additional resources? Once the browser has received the document (html file) it reads it. When a computer reads a file looking for something, it is called "**parsing**"

3. web browser reads html file to find resources

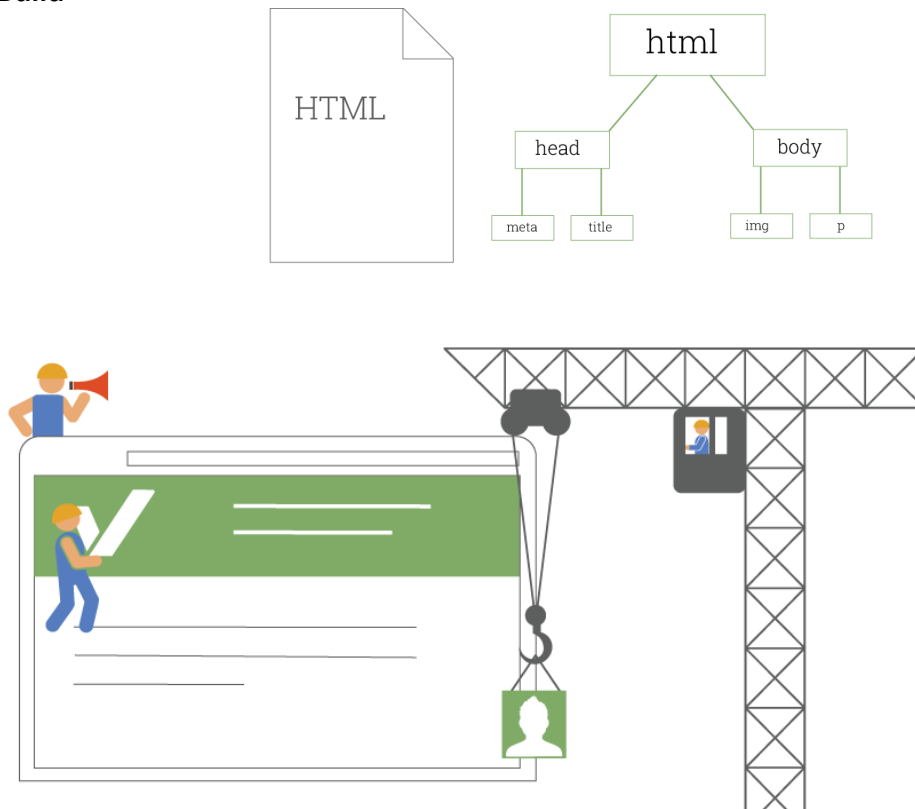
4. web browser requests found resources from server



The web browser looks at the entire HTML document and looks for any css, javascript and images that are referenced by the page. If resources are found in the HTML the web browser then requests each of those resource files from the webserver. The images, css, and javascript that the HTML file references will be downloaded by the browser.

The way it chooses to download those resources and the order in which resources are downloaded is a bit complex and will be explained in another article. If you are curious, I would look at the critical rendering path.

The Build



Once the web browser has the required resources it can start building the page. The way a web browser builds the page is by combining the information found in the document (the original HTML file) and the information found in the resources. There are basically three steps that the browser takes to build a page.

1. Build the DOM
2. Build the CSSOM
3. Build the Render Tree

Building the DOM

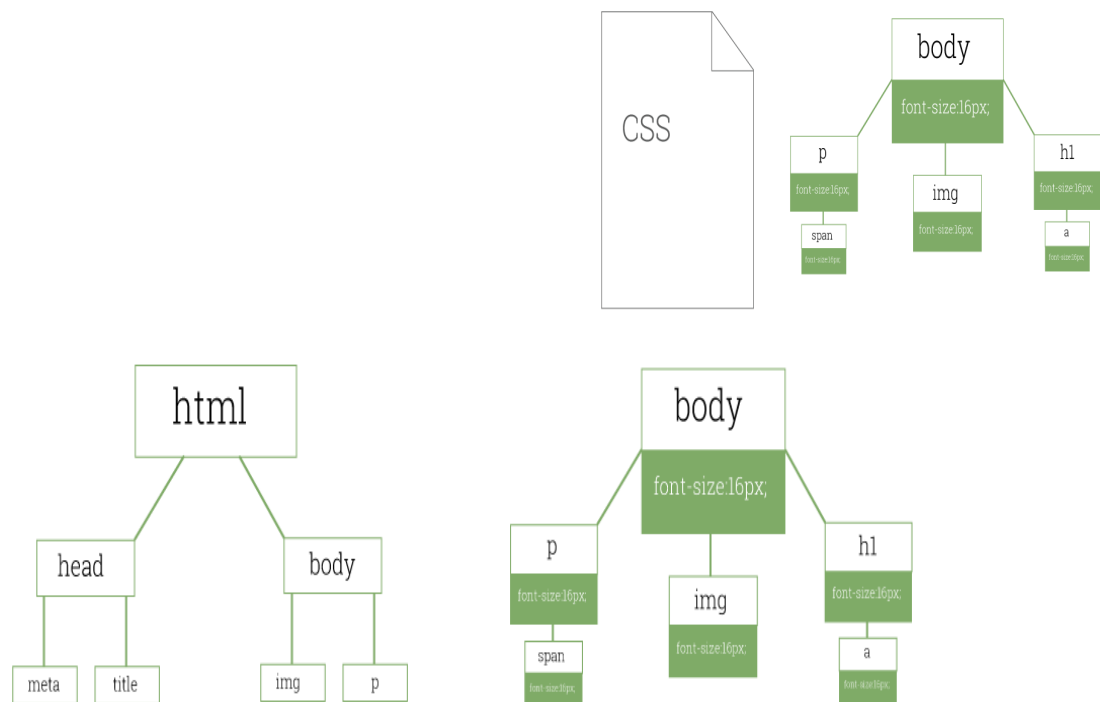
DOM stands for "Document Object Map". It is basically a map of where things are displayed on a page according to the HTML. The DOM represents what the HTML is saying by mapping out the page in a relational manner.

Building the CSSOM

CSSOM stands for "CSS Object Map". It is basically a map of what styles should be applied to different parts of the page according to the CSS. The CSSOM maps out the way things should be presented using styles.

Building the render tree

The render tree essentially takes the DOM and the CSSOM and combines them to create a full map of how the page will actually be laid out and painted.



The Render

After all of the above steps have been performed the browser can now finally put something up on the screen. There are two main things that happen here...

- Layout / Reflow
- Paint

Layout / Reflow

The browser at this point knows what it should display (the DOM) and in what manner to display it (the CSSOM) and the relationship between those two (the render tree).

What it does not know is the size in which to display everything and where everything will end up on a screen.

An example of this might be where a div called "sidebar" is supposed to be 25% of the screen on the right hand side.

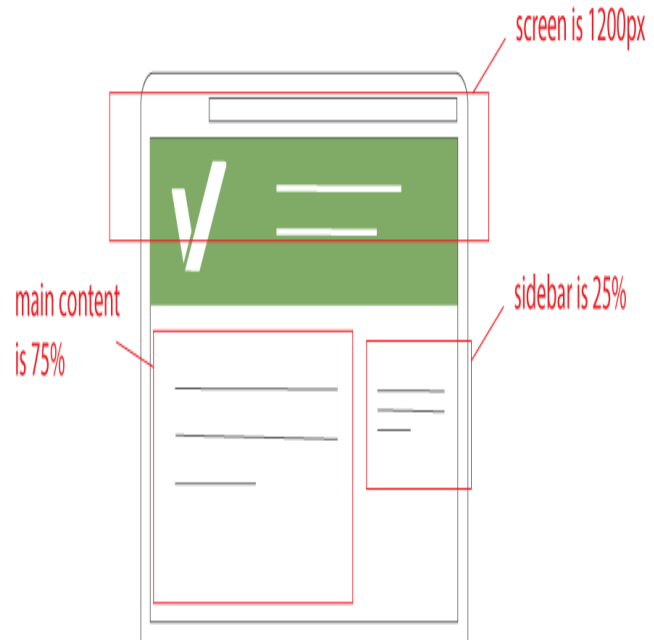
Hmmm, 25% of what?

This part is called the layout or the reflow, and it is basically where the browser determines how big a screen is and how that will affect the way the page is displayed.

Paint

Now that all the calculations are done, the browser can actually display something on the screen.

This is called the paint. In this final stage the browser will convert each node in the render tree to actual pixels on the screen.

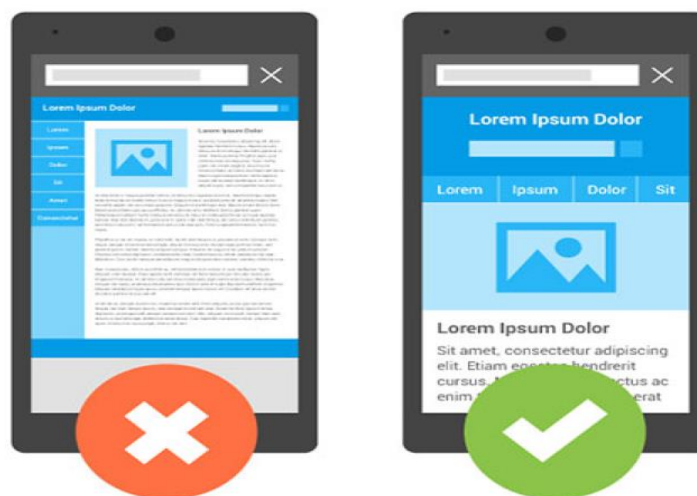


Putting it together

We broke the process of displaying a webpage into four parts...

1. Request
2. Response
3. Build
4. Render

Responsive Vs. Non Responsive Web Designs



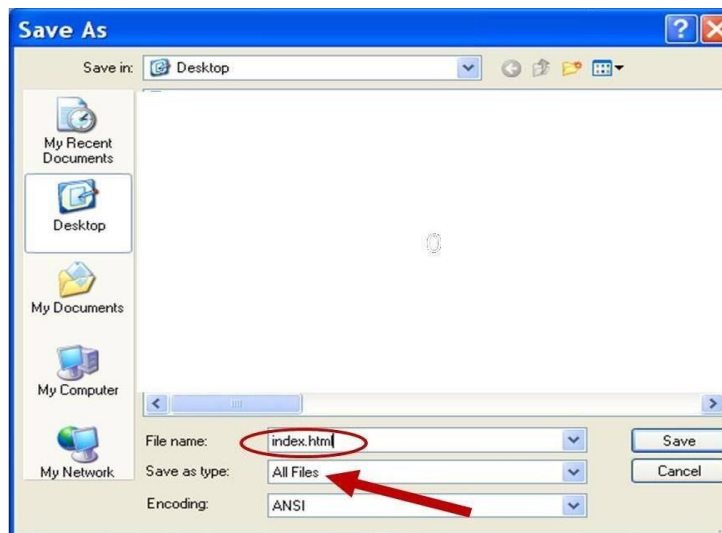
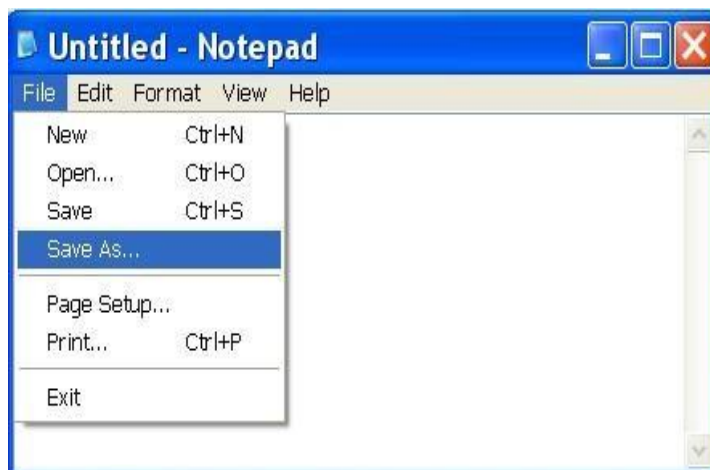
Introduction to HTML

1. Part 1-Creating directory/ index.html file

- In this part, we will create a directory in which we will save today's lab work. We will also learn how to create and save *index.html* file.
- On the desktop, create a folder named WELab01

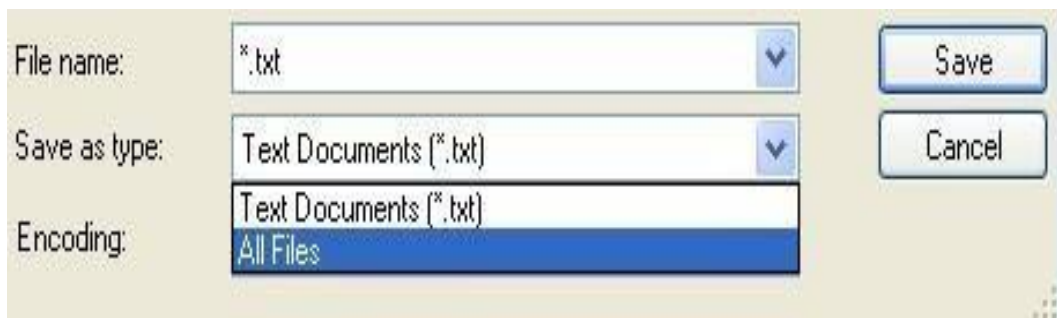
[Hint: this can be done by right clicking on the desktop, and then choosing New and then Folder]

- Create an index.html.
- Open the Notepad application. [Start > Programs > Accessories > Notepad]
- Save the empty file as index.html by clicking on the File menu of the Notepad.
- On the file menu, choose "Save As". The "Save As" window will appear as shown below:



Save the empty file as index.html on the Desktop. [We will use the Desktop as HTML playground, and once the files have been finalized, then they will be transferred to the WELab01 folder.] To save the file as index.html, index.html must be written as the file name with the “Save as type” set to *All Files*.

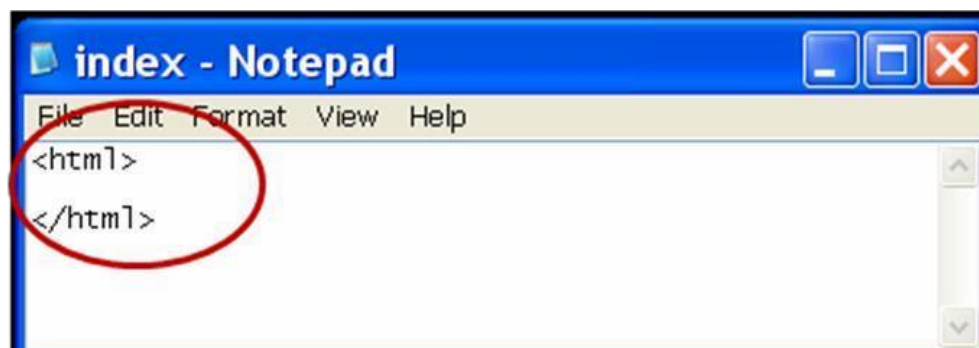
Note: As explained the “Save as type” should be set to *All Files*, if the “Save as type” field is set to *Text Documents (*.txt)* then the file would be saved as index.html.txt and not as index.html.



2. Part II: HTML Basics

a. Writing the HTML basic tags

Open the index.html file on the Desktop, and enter the `<html>` and `</html>` tags on the file. Note that in HTML, tags often (but not always) follow the convention that there is an opening tag and a closing tag. The opening tag in our context is `<html>` and the closing tag is `</html>`. These tags are used to inform the browser that the text between these tags is formatted according to HTML guidelines. The `<html>` tags are displayed in the figure below.



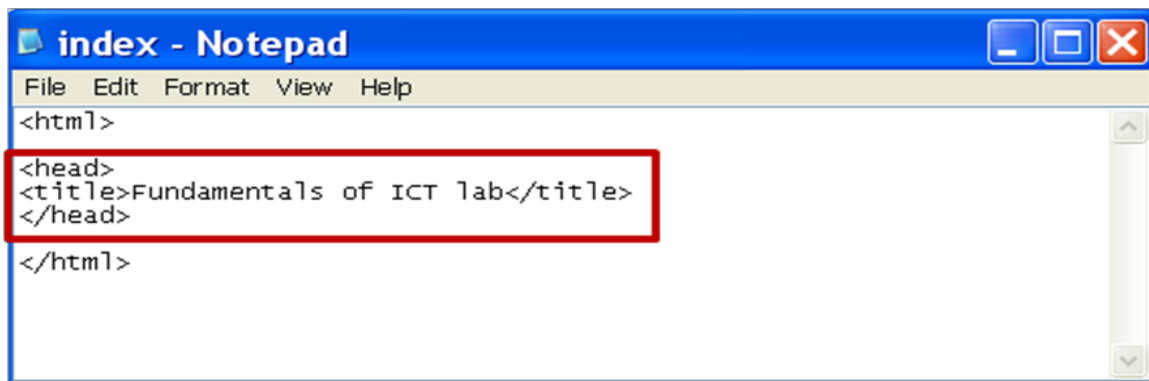
In the figure above, we do not have any HTML code in between the `<html>` tags. We will now start to code the HTML page.

b. Inserting the title of the web page

We will, first of all, learn the `<head>` tag and the `<title>` tag.

The **head** element (text between the *head* tags) contains general information, also called meta-information, about a document. The **title** (text between the *title* tags) is not part of the text of the document, but is a property of the whole document. It may not contain anchors, paragraph marks, or highlighting. The title may be used to identify the node in a history list, to label the window displaying the node, etc.

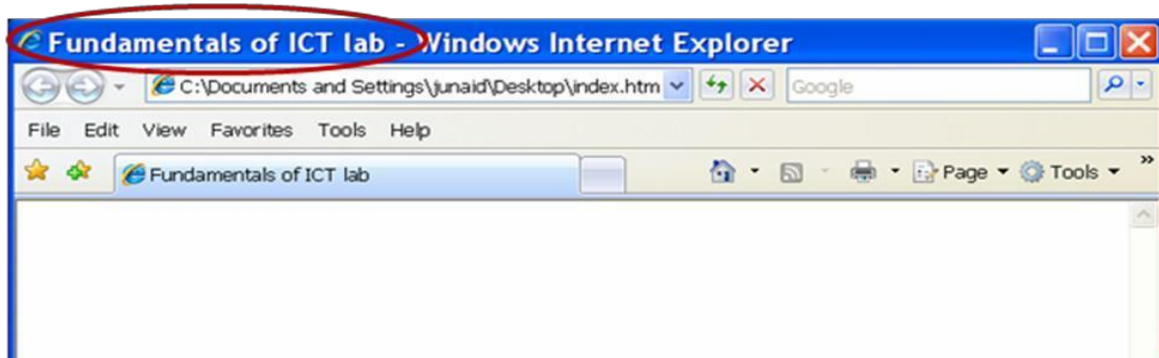
We will name the title of our webpage as Fundamentals of ICT lab. Note how the title text is enclosed in the **head** tags and the **title** tags



Open the index.html file on the Desktop (modified as above) with Internet Explorer (this can be done by simply double clicking on the index.html file). You will find that the Internet Explorer displays “Fundamentals of ICT lab” as the title of the web page as configured in the figure above.

The webpage is still empty as we have not added any text in the main page’s body yet (we will do that soon).

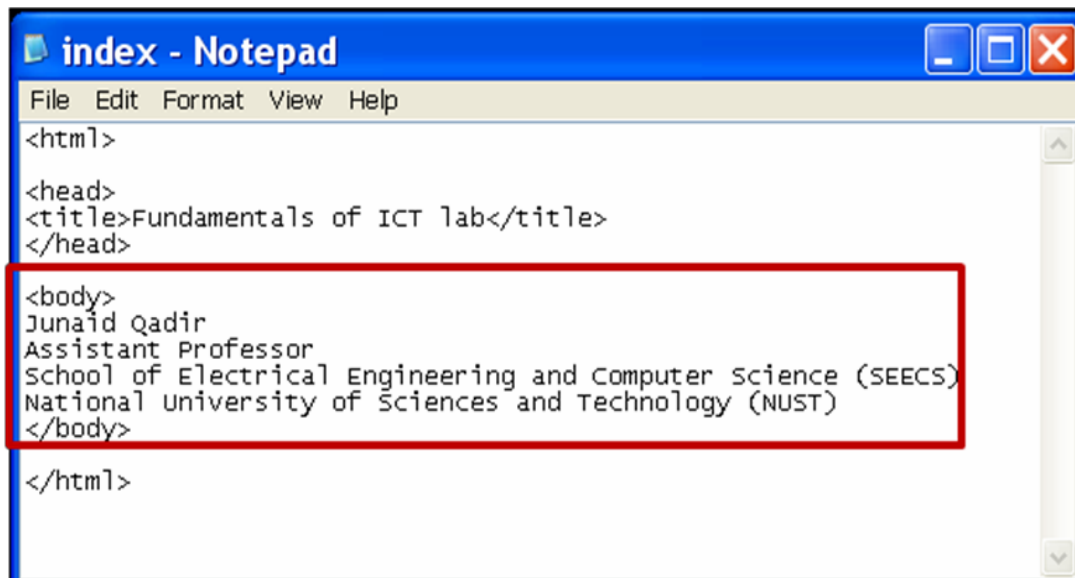
For now, the index.html page should look like the following.



Copy the index.html into the CFLab12 folder on Desktop and rename it as **Title.html**. [Hint: a file can be renamed by selecting, right clicking, and choosing the rename option.]

c. Inserting text in a HTML page

We will now add text to the main body of the HTML page. This is done using the **<body>** and **</body>** tags. The text contained between the **<body>** and **</body>** tags is displayed by a browser unlike data that is contained between the **<head></head>** tags. Add the text contained in the following figure in the index.html file on the Desktop. Save the file when done. The index.html file on Desktop should like the following.



```
<html>
<head>
<title>Fundamentals of ICT lab</title>
</head>
<body>
Junaïd Qadir
Assistant Professor
School of Electrical Engineering and Computer science (SEECS)
National University of Sciences and Technology (NUST)
</body>
</html>
```

After saving the index.html file on Desktop, open the file in a browser (by double clicking on the file). If the index.html file is properly configured, the browser display should be as follows. You will notice that all there is no line break, and all the text we had put between the **<body>** tags is on one line.

A browser has to be told by HTML tags where line breaks, new paragraphs, etc. should occur. Since we have not used those tags yet, all the text is on one line as shown in the figure below.

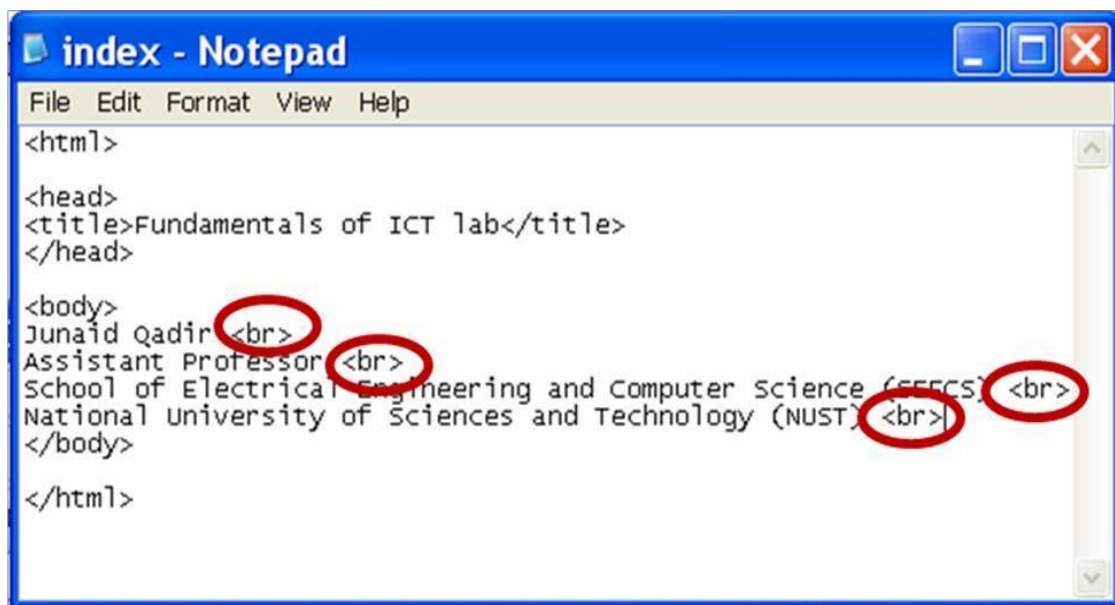


Save the index.html file on the Desktop in the WELab01 folder as **Allon1line.html**.

d. Inserting a line-break

We will now use the **
** HTML tag to introduce line breaks. This tag does not have a corresponding close tag and is used for introducing line breaks.

Open the index.html file on the Desktop and make the changes highlighted in the following figure.



After making the changes, save the index.html file and open it using a browser. The browser should display the page with proper line breaks as shown in the figure below. You will see that all the text is not on the same line any more.

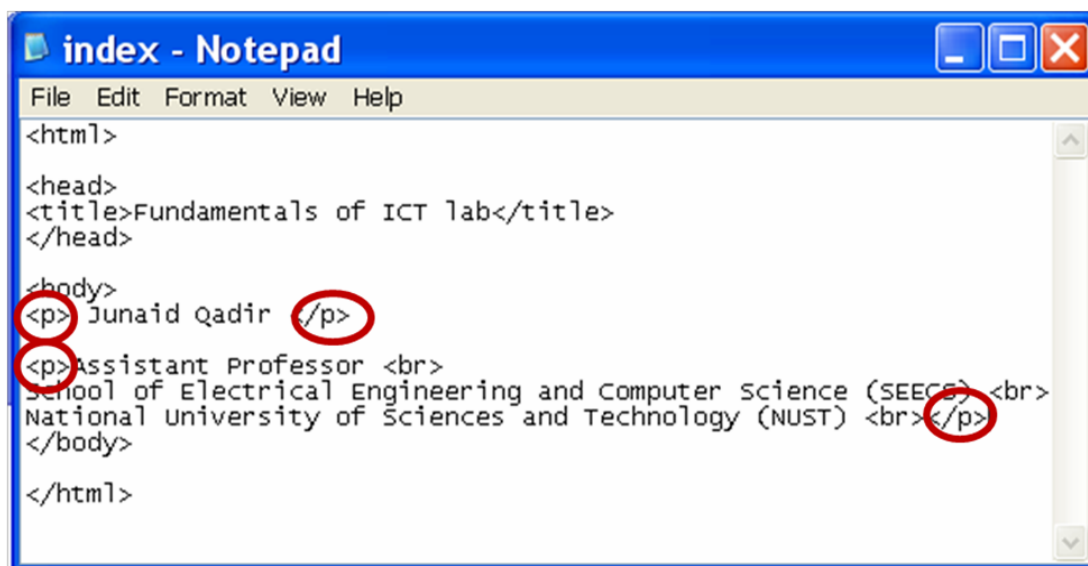


If you see the browser display as above, save the index.html file on the Desktop to the folder WELab01 as **Linebreak.html**.

e. Insert a paragraph

In the figure above, all the text is in a single paragraph (although separated by line breaks). To start a new paragraph, the HTML tag **<p>** should be used. The tag **<p>** is used to mark the start of a paragraph, and the end of paragraph is marked by **</p>**.

To observe how a paragraph is formatted in HTML, open the index.html file on the Desktop. Make the changes highlighted in the following figure and then save the file.



Open the index.html file on the Desktop (modified with the changes highlighted above) with a browser. You will see now that the web page is composed of two paragraphs. The second paragraph also has line breaks within the paragraph. The browser display should look like the following.

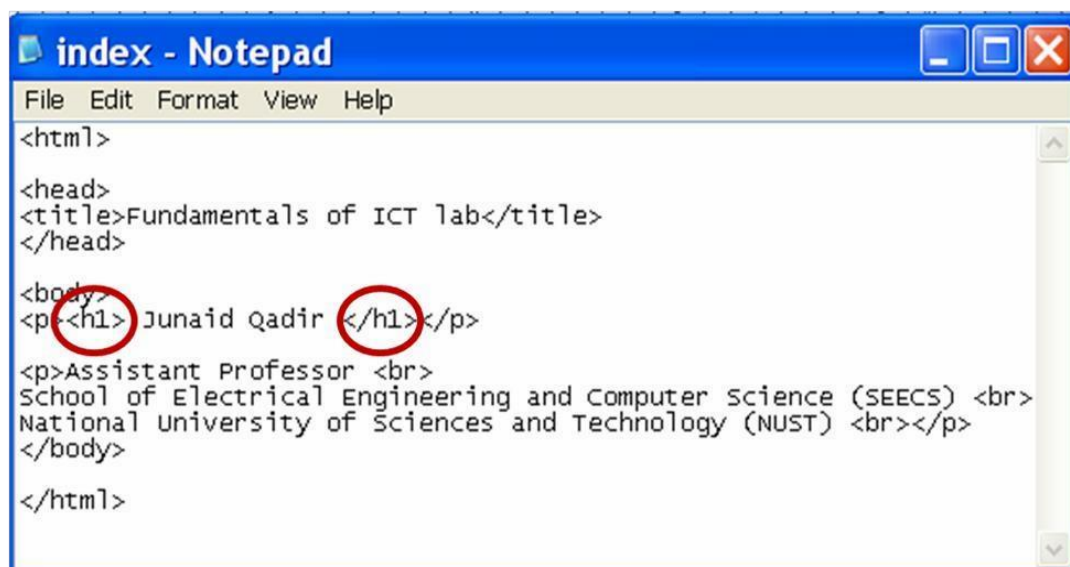


If you see the browser display as above, save the index.html page on the Desktop to the C:\Lab12 folder and name it as **Paragraph.html**.

f. Inserting headings

In the index.html file on the Desktop, currently no text is highlighted as a heading. If certain text is to be a heading, the HTML tags `<h1></h1>`, `<h2></h2>`, `<h3></h3>`, `<h4></h4>` (and so on) may be used. The tag `<h1>` is used for bigger sized heading with the heading size decreasing for `<h2>` `<h3>` and so on.

Open the index.html file on the Desktop and make the changes highlighted in the following figure. The first paragraph is highlighted using the `<h1>` tags.

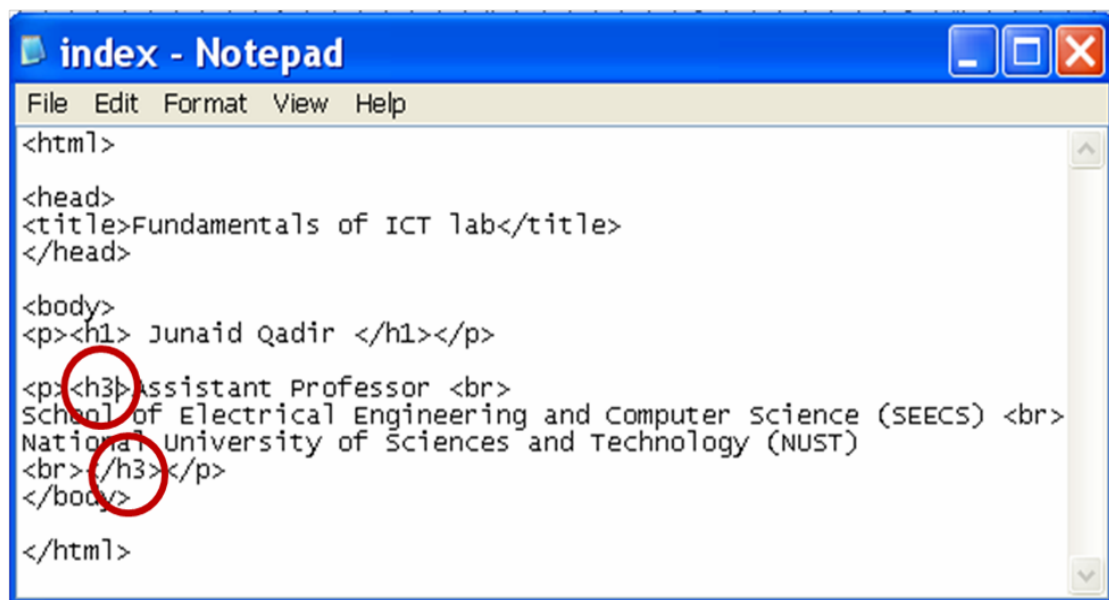


After making the changes in the index.html file on the Desktop as shown above, save the file. Open the file now with a browser. The browser display should look like the following. You will note that the first paragraph now highlighted as a heading with a large sized font.



If the browser display is as shown above, save the index.html file (on the Desktop) in the C:\Lab12 folder as **Headings1.html**.

Open the index.html file on the Desktop, and make the changes highlighted in the figure below.



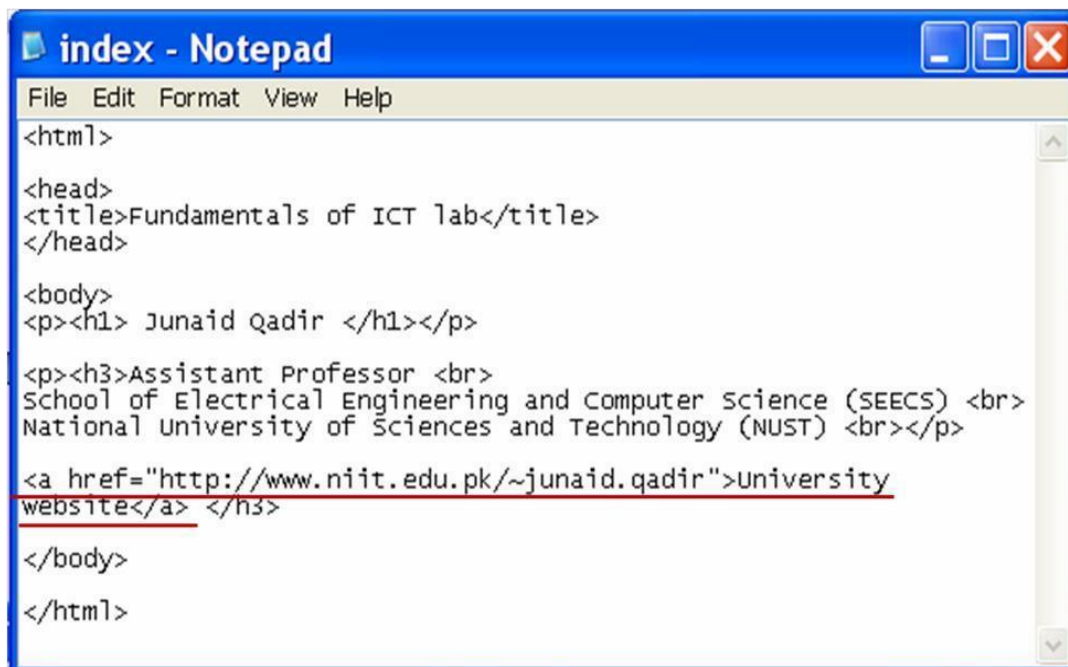
Save the index.html file after making the changes highlighted above, and open the file using a browser. You should now be able to see the second paragraph text highlighted as well as a smaller sized heading.



If the browser display is similar to as shown above, save the index.html file (on the Desktop) to the folder C:\Lab12 as **Headings2.html**.

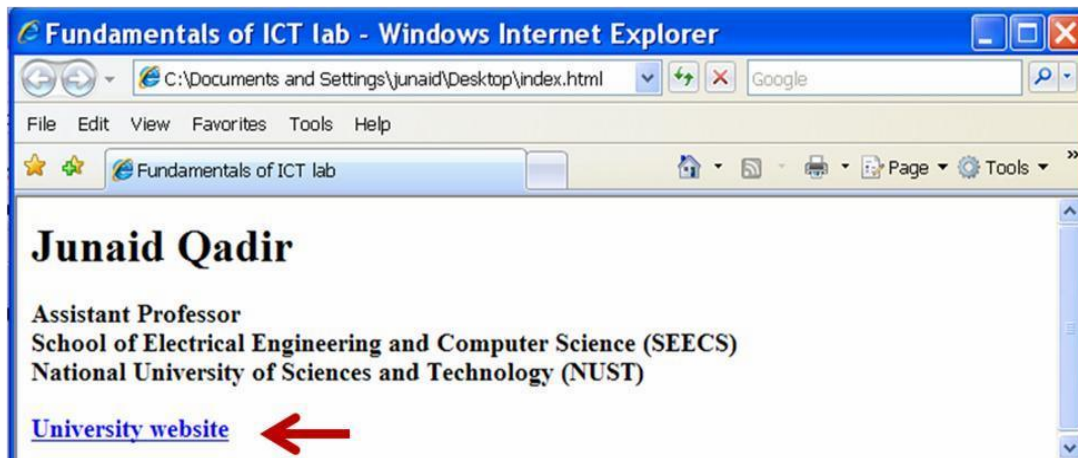
g. Inserting a hyperlink

We will now add a hyperlink on the index.html file. Open the index.html file on the Desktop and make the change highlighted in the following figure.



Save the file after making the change highlighted above. Open the file using a browser. You should see that the browser display is similar to as shown below. You will observe that a hyperlink to the specified URL is now added. Click on the URL and note where the target of

the hyperlink.

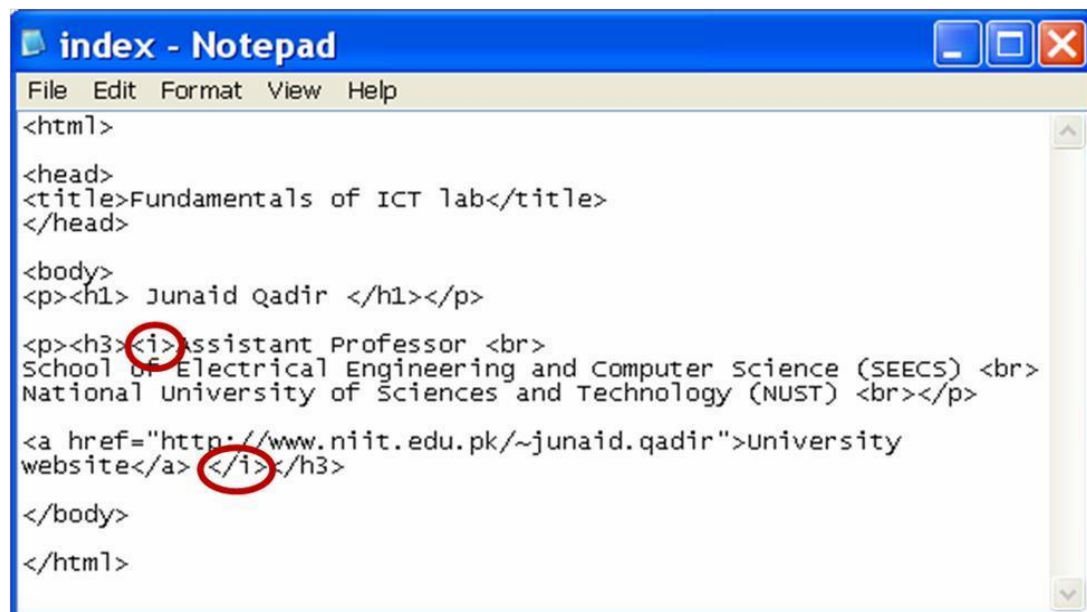


If the browser display is similar to as shown above, save the index.html file on the Desktop to the folder CFLab12 as **Hyperlink.html**.

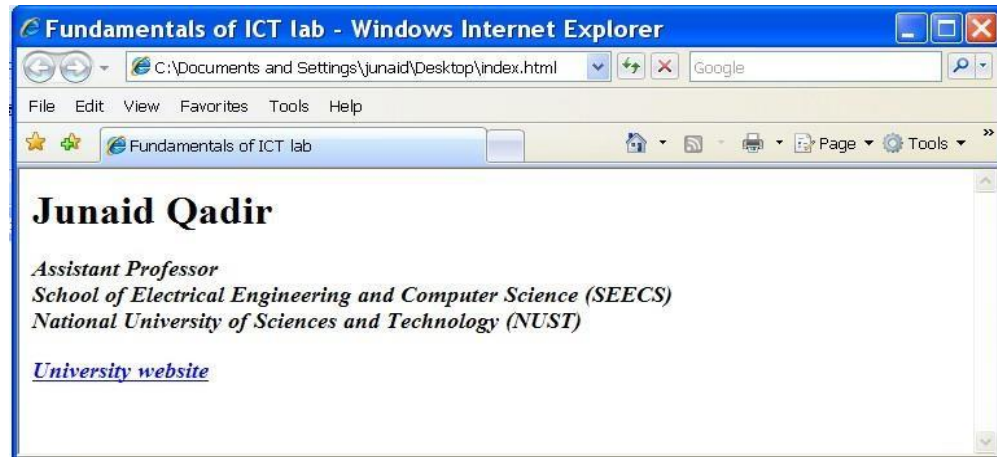
h. Making text italic, underlined, or bold

We will now see how a text can be *italicized*, underlined, or made **bold** in HTML. The tags for these tasks are, respectively, `<i></i>`, `<u></u>` and `<b></b>`.

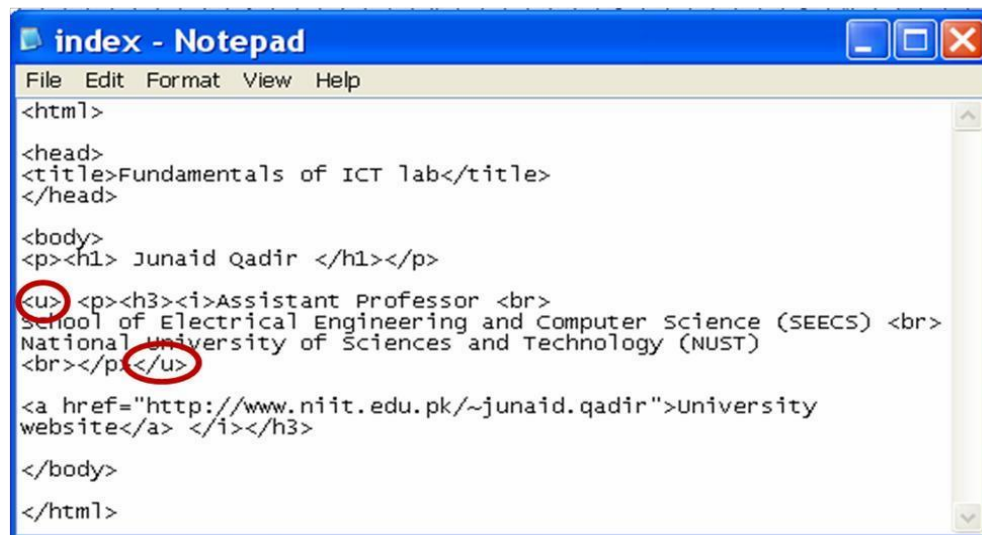
Italicizing: To italicize a body of text, it must be enclosed in `<i> </i>` tags. To demonstrate this, open the index.html file on the Desktop and make the changes highlighted in the following figure.



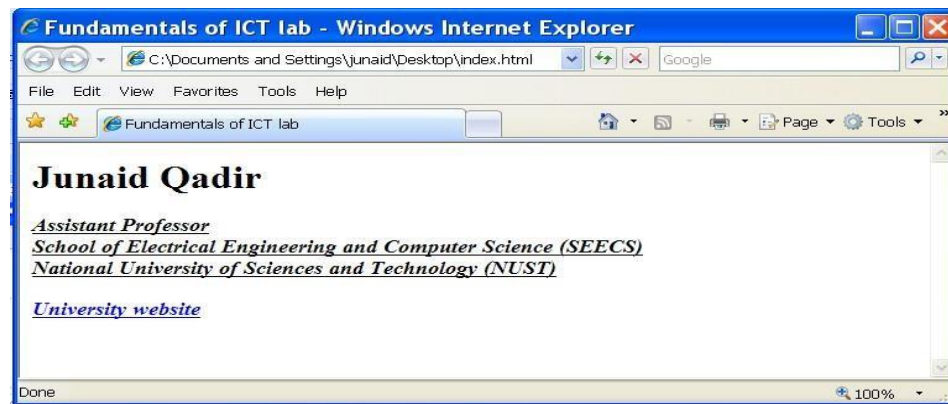
After making the changes shown above, save the file and then open it using a browser. You will observe that the text between the `<i>` and `</i>` tags is italicized. If the browser display is as shown below, save the index.html file in the WELAB folder as **Italics.html**.



Underlining: Similar to the approach shown above, use the `<u>` and `</u>` tags for underlining some text. Open the index.html file on the Desktop and make the changes as highlighted in the figure below.



After making the changes above, save the file and then open it using a browser. You should see the browser display similar to as shown below.



If the browser display is similar to as shown above, save the index.html file (on the Desktop) to the CFLab12 folder as **Underline.html**.

i. Lists

In HTML there is very little you can do without tables to precisely place any information on the page. HTML lists aid the programmer in defining steps or items on their own.

There are three types of lists available:

❑ Ordered Lists ❑ Unordered Lists ❑ Definition Lists

The opening and closing tags for the entire list are `<ol></ol>`, `<ul></ul>` or `<dl></dl>`. Each list item only has an opening tag which is `<li>`, unless using a definition list where you would use `<dt>` and `<dd>`.

An ordered list automatically inserts numbers sequentially for each item listed.

```
<ol>
<li>Stretch
<li>Warm-Up
<li>Jog for 30 minutes
<li>Cool Down
<li>Stretch
</ol>
```

Unordered lists follow the same type of format; however, the items in the list are in no order of importance. Unordered lists make use of round bullets before each item listed.

Definition lists are slightly more complicated to create; however, they do have the benefit of consistent formatting. Definition lists consist of a term and an interleaved and indented definition.

LAB TASKS

Task 1.

Discuss about different Web Servers.

Task 2.

Perform all tasks as demonstrated in lab.

Task 3.

Create your CV in HTML Format. Use the tags studied in this lab. You can also include additional html formatting tags.

What to Include in a CV

A CV should at least include the following:

- Your personal information
- Your work experience
- Your skills
- Your education
- Your personal profile and interests
- Your references